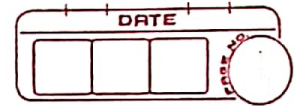


Assignment: 1



Q-1] Differentiate betn technical & non-technical problem with example

(a)

Technical Problem

- It requires technical knowledge, tool or computational method to solve it.

- Usually has clearly defined inputs, process & output

- Can often be solved using algorithm, software or engineering method

Eg: Designing an algorithm to find the shortest route b/w two location.

Non-Technical Problem

- It may involve human, social, emotional or organizational factors.

- May not have a single clearly defined solution.

- Often required communication, design decision making or behavioural change.

Eg: Resolving a conflict between two team members

(b)

What is mathematical thinking? How is it different from design thinking

⇒ Mathematical thinking is a way of solving problems using logic, reasoning, patterns, relationships, calculations & mathematical principles.

Design thinking is human centered approach to solving problems by understanding users, identifying their needs, generating ideas, creating prototypes and testing solution.

Mathematical Thinking

- It focuses on logic, number, pattern and reasoning.

- usually seeks a logically correct answer

- uses mathematical models and reasoning.

Design Thinking

- Focuses on people needs, creativity and usability

- Often explores multiple possible solution.

- Use empathy, ideation, prototype & testing

Q-2] Example of social problem addressed using computational Techniques
 ⇒ Traffic congestion (problem) is a major social problem in cities

A computational solution can use traffic data, GPS information, sensors and algorithms to analyze traffic conditions and suggest alternative routes.

for example: A navigation system can collect information about vehicle speed & road conditions and use algorithms to recommend the shortest or fastest route.

Result: Computational technique can reduce travel time, improve traffic management & help reduce fuel consumption.

(b) Name & briefly describe two key components of computational thinking
 ⇒ i) Decomposition: It means breaking a complex problem into smaller & manageable sub problems.

Eg: For an online shopping system, product search, shopping cart, payment, tracking orders etc.

ii) Pattern recognition: It means identifying similarities, trends or repeated structures in a problem.

Eg: If a traffic system observes that a particular road is congested every morning betⁿ 8-10 am, this repeated pattern can be identified & used for better traffic management.

Q3) How is computational thinking related to CS and programming?

⇒ Computational thinking is a problem solving approach that involves breaking problem with identification, creating abstractions and developing algorithms.

Computer science is a broad field that studies computation, data, algorithm, software, hardware & information processing.

Programming is a process of implementing an algorithm or computational solution using programming language such as C, C++, Java or Python.

Relationship: Real world problem → computational thinking → Algorithm → programming → computer based solution.

Eg: To create a programme that calculates student grades

- (1) Break the problem into smaller task.
- (2) Identify the required data.
- (3) Design an algorithm to calculate grades.
- (4) Implement the algorithm by using C/Python.
- (5) Test the program & correct errors.

Thus computational thinking provides problem solving approach. Computer science provides concepts and programming implements the solution.

(b) Explain computational & informational aspects of real world problem with an example.

⇒ consider problem of college attendance management.

The computational aspect deals with the processing of operations performed on the data.

- (1) Calculate attendance percentage
- (2) Determine whether the student has sufficient attendance
- (3) Generate attendance reports
- (4) Sort students according to attendance
- (5) Identify students w/h attendance below a particular limit

The Informational aspect deals with what information is collected, stored and communicated.

for eg. student name, roll no., date etc.

Therefore, the computational aspect processes the information, while the informational aspect concerns the data of information being processed.

Q4) Compare, divide & conquer with mapping to different problem as computational problem solving approaches. when will one be more suitable than other

⇒

Divide & Conquer	Mapping to diff. problem
• Breaks one problem into smaller part	• Relates one problem to another problem.
• sub problems are usually similar to original problem.	• The mapped problem may be structured different
• Suitable for large problem divided independently	• Suitable when a known problem provides effective solution.
Eg: merge sort	Eg: transforming a problem into known graph problem

A traffic management system can computationally process logical components, but human behaviour & unpredictable events make some aspects difficult to model perfectly.

Q5) Consider a problem like sorting a large dataset would a divide and conquer approach be better than an iterative method?

Justify your answer

⇒ For sorting a large dataset, a divide and conquer approach can be better, especially when using algorithm such as mergesort.

The reason is :-

- (1) The large dataset can be divided into smaller parts.
- (2) Each smaller part can be sorted independently.
- (3) The sorted parts can then be combined.
- (4) The approach is efficient for large dataset.
- (5) Sub-problems can potentially be processed in parallel.

Eg. Merge sort as follows -

for n elements, merge sort has a time complexity of approximately $O(n \log n)$. Therefore, for large datasets, divide & conquer is often preferable to a simple iterative approach, particularly when efficiency and scalability are important.

(b) Design a computational approach for a task scheduling system in an organization. Identify repetitive & non-repetitive aspects and propose a solution.

⇒ Consider an organisation where different employee have different task & deadlines.

Step 2: Input Information

The system collects employee ID, task name, Task priority, deadline, estimated completion time, employee availability, task dependencies.

Step 2: Repetitive aspects are checking employee availability, assigning tasks, checking deadlines, updating task status, sending reminders, generating daily/weekly reports. These activities can be automated using a computer programme.

Step 3: Non-repetitive aspects are the some decisions may require a human judgment like handling unexpected emergencies, resolving conflict between employees, changing priorities because of special circumstances, deciding which task is strategically most important.

Step 4: Proposed computational solution use an algorithm such as
 Input task → check priorities → check deadline → check employee availability → Assign task → monitor program → send reminder → update schedule

A computational task scheduling system can be automate repetitive scheduling activity while following manager to handle unusual or complex situation. This improves efficiency, time management and productivity.

Go
17/11/26
Cate